Lab 2: Fine-tune vs prompt

Does fine-tuning beat your best prompt?

Aarne Talman

5 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

The question

Lab 2

For *your* task, does fine-tuning beat your best prompt?

You compare two ways of getting a model to do your task:

- **prompting**: give the model instructions and, optionally, a few examples, without changing it
- **fine-tuning**: train the model on your labelled examples

Both are scored on the same test examples, which were not used for anything else.

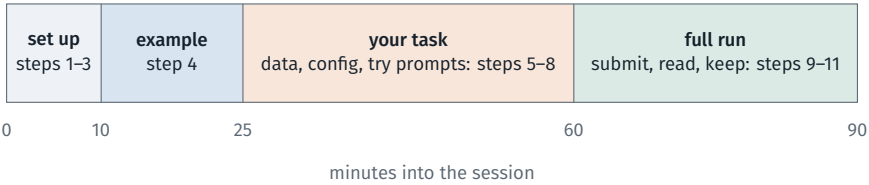
For the final report

- one results table (`summarize_results.py` prints it)
- one loss plot
- one paragraph: was fine-tuning worth the extra time and GPU use?

Minimum: README steps 1–11 once, with default settings. Ideas for going further are in the README.

Everything you need is in `Week-2/Lab-2/README.md`, step by step.

Today



The steps are those in the README. Start with the **smoke test** (step 2): it checks the scripts without a GPU. If it has not passed by minute 15, put your hand up.

The full experiment is **one GPU job**. It scores the prompts, trains the model and runs the general-knowledge check, and it can take an hour or more. Submit it by minute 60, even with a first version of your prompt.

Jobs run in the normal Roihu queues, so they may wait before they start. They keep running after the session; check them later with `squeue --me`.

Your data and your prompt

```
# 1. download an existing labelled dataset and split it (README step 5)
python3 from_huggingface.py --dataset stanfordnlp/sst2 --out sst2.jsonl
python3 make_splits.py --input sst2.jsonl --fields sentence \
    --label-field label --out data/my-task

# 2. lab2lib/prompting.py: the two settings you change (step 6)
TASK_FIELDS = ("sentence",)
TASK_SYSTEM = ("Is the review positive or negative? "
               "Use exactly one of: positive, negative. "
               "End with 'Label: <label>'.")
```

Three splits of your data

- **train**: what the model learns from (1,000–5,000)
- **dev**: for choosing the prompt and settings
- **test**: only for the final scores

You do **not** create a dataset. Use an existing labelled one: SST-2 (above), another dataset listed in the README, or your own file.

Keep the last sentence of the instruction: the scorer looks for `Label: <label>`. In step 8 you try the instruction on dev examples with `try_prompt.py` and improve it.

The experiment

One GPU job does everything

You prepare the data and config (README steps 5–7) and try your prompt interactively on dev examples (step 8). Then you submit **one GPU job** (step 9), which runs the five phases below in order, and read the results (step 10).

1. Prompting	your instruction in four versions: base and instruct model, each with no examples and with 5 examples; the best on dev is *the* baseline
2. Knowledge, before	the instruct model answers 50 general-knowledge questions
3. Fine-tuning	LoRA trains a small add-on to the instruct model on your training data
4. Testing	the best prompt and the fine-tuned model are scored on the same test set
5. Knowledge, after	the fine-tuned model answers the same 50 questions

Your conclusion (step 11): does fine-tuning beat the best prompt, and at what cost?

Read the outputs before you trust a score: a score of exactly 0 or 100 usually means the scorer is wrong, not the model (`show_examples.py`).

If you have time: tune the training

Setting	Start	What it controls
learning rate	0.0002	how large each training step is
epochs	2	how many times the model sees the training data
LoRA rank	16	the size of the adapter

Change *one* setting at a time, give each run its own name, and keep a small table of the runs. The README lists other extensions.

Lower training loss is not a result

The **loss** measures how wrong the model's answers are. If the training loss falls while the dev loss rises, the model is **memorising** the training examples.

Then try fewer epochs or a lower learning rate. Choose settings on the dev loss, never on the test score.

Checklist for today

[ ] smoke test passed (step 2)
[ ] example job finished on `gputest` (step 4)
[ ] dataset downloaded and split (step 5)
[ ] prompt adapted (step 6)
[ ] config made, `check_data.py` clean (step 7)
[ ] prompt tried on dev with `try_prompt.py` (step 8)
[ ] full job submitted to `gpumedium` (step 9)
[ ] table printed with `summarize_results.py`
[ ] ten raw outputs read with `show_examples.py`
[ ] results folder and config kept for the report (step 11)

Good practice on Roihu

- follow a job with `squeue --me` and `tail -f logs/lab2-lora-<jobid>.out`
- do not run models on the login node
- leave interactive GPU sessions with `exit` when you are done, and close unused GPU Jupyter sessions

Stuck for more than ten minutes? Put your hand up. The README has a troubleshooting table.